구현 설명서 / 연구와 적용의 연결

요구사항을 지키는 개발 하네스

외부 실행 제어와 구체적 명세로 임의 범위 축소를 줄이는 구조

사용자가 요청한 기능을 빠뜨리지 않고 구현하고, 개발 중 발견한 하네스 문제를 실제 사용 피드백으로 보완하기 위한 설계다. 이 문서는 현재 작업공간에 반영한 코드와 운영 지침을 설명하고, 관련 논문·공식 기술 자료·커뮤니티 경험담을 각 설계에 연결한다.

반영한 핵심

요청 원문을 계약과 수용 시나리오로 구체화하고, 별도 실행기가 개발·검증·완료 상태를 관리한다. 모델은 구현과 검토를 수행하며 필수 요구사항을 스스로 삭제하거나 공식 완료 상태를 확정하지 않는다.

현재 결과	확인 범위	다음 평가 방식
하네스 관련 29개 파일을 로컬 활성 작업공간에 반영	문법·import·JSON/TOML·CLI 도움말 연결 확인	사용자가 실제 개발을 진행하며 체감·수정 요청

구현 반영과 효과 입증은 구분한다. 이번 변경에서는 사용자 지시에 따라 모델 실행, 행동 테스트, 비교 실험을 수행하지 않았다. 범위 축소가 얼마나 감소했는지, 비용·속도가 얼마나 달라졌는지는 아직 측정하지 않았다.

읽는 순서: 2쪽 비교 · 3쪽 요구사항 · 4-5쪽 실행/완료 · 6쪽 개선 흐름 · 7쪽 연구 연결 · 8쪽 사용 경로 · 9쪽 평가와 한계 · 10쪽 원문 링크

01 / 문제와 선택

사용자 방식과 조사한 방식의 비교

사용자가 제기한 핵심은 세 가지였다. 추상적인 요청에서 바로 구현을 시작하고, 구현 도중 필수 기능을 임의로 줄이며, 그 원인이 하네스에 있어도 고쳐지는 흐름이 없다는 점이다. 따라서 개발자를 더 추가하기보다 요청 보존과 실행 권한부터 분리했다.

구분	기존·검토한 접근	이번 반영
요구사항 준비	운영자가 기획과 작업 브리프를 정리하지만, 자연어 지침 준수에 의존	원문 인용과 필수 요구·시나리오를 연결. 중요한 질문이 남으면 개발 대기
외부 운영자	별도 상위 AI가 작업을 분해하고 순서·완료를 판단	현재 대화는 기획, 외부 프로그램은 단계 실행과 공식 완료 판정
검증과 범위	개발자 보고·상태 제출 후 운영자가 검토	실제 명령 결과와 독립 검토 수집. 계약 변경은 사용자 메시지에 연결
하네스 수정	기존 개선 절차가 자기 변경에도 재현·평가·승인을 요구	이번 직접 수정은 사용자 지시로 기존 절차를 적용하지 않고 반영. 이후 실제 사용 피드백으로 보완

다른 AI를 붙이는 것만으로는 충분하지 않다

Reddit 사례는 단계 생략과 비결정적 라우팅 문제를 이유로 코드 기반 워크플로로 전환했다고 설명한다. 소프트웨어 개발에 LangGraph를 적용했다는 댓글도 있다. 다만 모두 작성자의 경험담이며, 해당 구조의 보편적 효과를 증명하는 실험은 아니다. [1]

반면 Magentic-One은 관리 에이전트와 작업·진행 원장을 활용한 논문이다. 계획과 진행을 분리해 관리하는 근거는 되지만, 관리 AI에게 완료 권한을 주는 것과 외부 코드가 완료를 제한하는 것은 다른 선택이다. 이번에는 원장 개념을 참고하고 최종 상태는 프로그램이 갖도록 정했다. [4]

02 / 요구사항 구체화

구현 전에 무엇을 확정하는가

예를 들어 "번역을 편집할 수 있게 해줘"라는 요청이 오면 바로 화면과 저장 API부터 만들지 않는다. 프로젝트 정책에서 대상·권한·수정 보호·이력·동시 수정 규칙을 먼저 읽는다. 이미 정해진 정책은 재질문하지 않고, 중요한 미정 선택만 사용자에게 묻는다.

계약 항목	기록할 내용과 역할
`source_coverage`	사용자 원문의 실제 인용을 요구사항 ID에 연결. 요약 과정의 누락을 추적한다.
`requirements / scenarios`	필수 동작과 관찰 가능한 정상·실패·경계 조건. 예: 동시 수정 시 기존 값을 조용히 덮어쓰지 않는다.
`questions / assumptions`	미정 질문과 가정의 확인 상태. 중요한 미정 사항이 있으면 WAITING_REQUIREMENTS.
`owned_paths / constraints`	수정 소유 범위와 기존 도메인 정책. 요청한 일을 임의로 "후속 개발"로 제외하지 않는다.
`verification_commands`	필수 요구사항에 실제 검증 명령을 연결. 기존 보호 테스트/설정은 별도로 동결한다.

두 번의 누락 검토가 필요하다

첫 번째는 원문 → 계약의 누락이다. 원문에 10개 조건이 있는데 계약에 7개만 남으면 7개 테스트가 모두 통과해도 부족하다. 별도 읽기 전용 명세 검토가 원문 전체와 계약을 비교한다. 두 번째는 계약 → 구현의 누락이며, 개발 이후 실행 결과와 독립 코드 검토로 확인한다.

Anthropic은 짧은 요청을 명세로 확장하는 planner와 독립 evaluator를 설명하며, planner가 없을 때 작업 범위가 작아지는 현상을 보고했다. 이번에는 이 역할 분리를 참고하되, 추가 기능을 무조건 늘리는 대신 사용자가 요청한 범위와 중요한 가정을 명확히 하는 데 초점을 맞췄다. [2]

Spec Kit도 명세를 계획·구현으로 이어가는 구조화된 프로세스를 제공한다. 이는 관련 방식의 대조 자료이며, 이번 구현에 Spec Kit 패키지나 명령 체계를 설치한 것은 아니다. [9]

03 / 실행 구조

기획은 대화에서, 전환은 코드에서



외부 실행기는 Python 표준 라이브러리와 SQLite로 작성했다. 사용자는 계속 현재 대화에서 요청과 결정을 전달한다. 운영자는 별도 개발 worktree를 준비하고 실행기를 호출한다. 실행기는 단계마다 별도 Codex 프로세스를 시작하고 결과를 기록한다.

책임	담당	구체적인 권한
목표·정책·범위 결정	사용자 + 운영 대화	원문·후속 결정 보존, 중요한 미정 사항 해소
구현 방법 선택	개발 세션	계약과 소유 경로 안에서 코드 수정
명세/구현 의미 검토	별도 읽기 전용 세션	누락·실제 동작에 대한 근거 제출
실행·재작업·완료	외부 프로그램	필수 단계를 실행하고 모든 조건 충족 시 VERIFIED
병합·배포	별도 운영 결정	실행기는 자동 merge/push/배포를 수행하지 않음

구현의 자율성과 완료 권한을 분리했다

Anthropic의 workflow/agent 구분과 Microsoft의 결정론적 워크플로 설명은 모델이 단계 안에서 일하되 전체 흐름은 코드가 정의하는 방식을 제시한다. 이번 구조도 개발 방법은 모델이 정하고, 필수 검토·실패 처리·완료 전환은 프로그램이 정한다. [6] [7]

LangGraph 문서의 노드·공유 상태·전환 개념도 참고했다. 다만 현재는 단일 로컬 프로젝트에 필요한 최소 실행기를 직접 구현했다. LangGraph·Temporal·Azure 서비스를 실제로 도입하거나 분산 내구성을 확보했다고 주장하지 않는다. [8]

제어 파일의 위치도 분리

상태·원문·계약·실행 런타임 복사본을 개발 workspace 밖에 둔다. 개발은 쓰기 가능한 별도 세션, 검토는 읽기 전용 세션, 검증은 별도 scratch를 사용하는 제한 프로세스로 구성했다. 이 경계의 실제 작동은 이번에 실행 검증하지 않았다.

04 / 완료와 실패

"완료했습니다"를 상태로 인정하지 않는다

개발자의 최종 응답은 제출물이다. 실행기는 실제 프로세스의 성공 종료, 검증 명령 결과, 독립 검토, 계약·코드의 일치 여부를 확인하도록 작성했다. 테스트 실패를 검토 모델의 좋은 평가로 덮을 수 없고, 지난 코드에서 얻은 결과로 새 코드를 완료 처리하지 않는다.

완료 조건	기록·확인하는 근거
구현 전 준비	중요 질문/가정 해소, 필수 요구·시나리오 연결, 독립 명세 검토 PASS
실제 실행 성공	명령 argv, 종료 코드, 시작/종료 시각, timeout·취소 여부, 로그 지문
요구 충족 검토	필수 requirement_id / scenario_id 전체에 근거. 실제 명령 ID 또는 관찰 명령 인용
결과의 최신성	원문·계약·동결 파일·런타임 지문. 검증 당시 코드 지문과 현재 상태 비교
소유 범위 유지	개발 전후 파일 목록 비교. 소유 범위 밖 변경을 발견하면 BLOCKED

실패해도 요구사항은 유지한다

검증 실패는 같은 계약을 유지한 재작업으로 돌아간다. 기본 재작업 한도는 2회이며 계약에서 정한다. 한도 소진은 성공이 아니라 BLOCKED다. 중단된 실행은 resume으로 이어가고 취소는 CANCELLED로 남긴다. 실패나 취소 때문에 제품 파일을 자동으로 되돌리지는 않는다.

사용자 결정으로 범위가 바뀌면 최초 원문을 덮어쓰지 않고 후속 메시지, 이전/새 계약 지문, 새 버전을 기록한다. 이 변경은 과거 명세 검토와 구현 PASS를 무효화한다. 개발자가 검증 기준을 스스로 낮추는 경로와 구분하기 위한 장치다.

LangChain의 완료 전 체크와 build-verify 반복은 조기 종료 문제에 대한 참고 자료다. 이번에는 "검증하라"는 리마인더를 넘어 실제 검증 실행과 상태 판정을 외부로 옮겼다. 글의 벤치마크 개선이 이 추가 선택의 효과까지 증명하는 것은 아니다. [3]

QUICK의 낮은 영향 문구·스타일에는 가짜 테스트를 만들지 않는다. 명령이 없는 경우에도 별도 검토 세션의 실제 파일 관찰을 요구한다. FULL은 모든 필수 요구사항에 검증 명령을 연결한다.

05 / 사용 중 개선

이상현상은 근거를 남기고 원인을 나눈다

앞으로 실제 개발에서 "요청한 화면이 빠졌다", "중요 질문 없이 구현했다", "같은 수정을 반복한다"는 현상이 보이면 운영자가 작업 ID와 관찰 근거를 연결해 기록한다. feedback은 사용자의 관찰을 보존하는 경로이며, 입력만으로 하네스를 자동 수정하지 않는다.

관찰	먼저 구분할 원인	후속 대응
기능 누락	원문→계약 누락 / 계약→구현 누락	해당 단계의 요구 연결·명세 검토·구현 검토를 보완
검증 실행 실패	의존성·권한·로컬 환경 / 제품 결함	환경과 제품을 먼저 확인. 곧바로 하네스 문제로 확정하지 않음
반복·조기 종료	부실한 피드백 / 재작업 제어 / 미정 요구	실행 이력과 종료 사유를 비교해 수정 요청을 구체화
과도한 절차	QUICK 범위에 불필요한 기획·질문	이미 받은 결정 재사용, 필요 역할과 산출물 축소

AHE는 하네스 구성요소, 실행 경험, 수정 결정의 관찰 가능성을 연결한다. 이 관점을 가져와 원본 관찰과 수정 이유를 추적하도록 했다. 하지만 NexAU 기반의 무인 진화 알고리즘, 벤치마크 반복 채택, 성능 상승률을 이 프로젝트에서 재현한 것은 아니다. [5]

이번 하네스 수정에는 기존 절차를 적용하지 않았다

사용자는 수정 대상 하네스를 다시 개발·검증의 기준으로 삼는 방식에 이의를 제기했고, 기존 하네스를 무시하고 계획을 완전히 반영한 뒤 직접 개발해 보며 평가하라고 지시했다. 이에 따라 이번에는 별도 비교 평가나 행동 테스트를 완료 조건으로 만들지 않았다.

일반적인 자동 생성 후보에 대한 기존 사람 승인 경계는 남겨 두었다. 다만 사용자가 하네스 자체를 직접 수정하도록 지시한 경우에는 그 최신 범위와 절차를 우선한다. 이번 지시는 앞으로의 모든 자동 후보나 배포에 대한 일괄 승인으로 기록하지 않았다.

AHE가 보고한 회귀 예측의 한계는 "좋아질 것 같다"는 설명만으로 효과를 확정하면 안 된다는 근거다. 이번 사용자 사용 평가 역시 유용한 운영 피드백이지만 통제 실험에 의한 인과 효과와 같지는 않다. [5]

06 / 자료와 구현의 연결

어디에서 가져왔고, 어디까지 반영했는가

아래 연결은 자료를 그대로 복제했다는 뜻이 아니다. 원문의 아이디어를 현재 요구에 맞게 적용한 부분과, 설치·재현하지 않은 부분을 함께 표시했다. 각 번호는 원문으로 연결된다.

연구·자료	참고한 관점	이번 적용 / 적용하지 않은 것
커뮤니티 사례 [1]	AI의 판단과 실행 순서 통제 분리	Python 상태 머신을 기본 경로로 채택 / 사례의 실패율을 로컬 성과로 사용하지 않음
Anthropic 장기 개발 [2]	구현 전 planner, 독립 evaluator	원문→계약 검토와 계약→구현 검토 분리 / 원문의 제품 확장 전략을 일괄 강제하지 않음
LangChain 하네스 글 [3]	실행 trace, 완료 전 확인, 수정 반복	실행 영수증·로그·제한 재작업 / 전체 LangSmith trace나 middleware 패키지 도입은 아님
Magentic-One [4]	작업 사실·가정·진행 원장	계약과 상태·사건 이력 분리 / 논문의 전체 다중 에이전트 알고리즘은 미도입
AHE [5]	관찰·원인·수정 결정 연결	feedback과 근거 보존, 기존 개선 이력 연결 / 무인 진화 실험은 미실행
Workflow 공식 자료 [6] [7] [8]	코드가 정한 흐름 안에서 모델 실행	단계·전환·필수 조건을 외부 구현 / 각 프레임워크·클라우드 서비스는 미설치
Spec Kit [9]	명세를 개발까지 이어가는 구조	요구사항 방식의 관련 대조 자료 / SDK·명령 체계는 가져오지 않음

숫자를 읽는 기준

Reddit의 약 40% 문제 발생에서 3개월간 0% 오케스트레이션 실패로 바뀌었다는 주장은 콘텐츠 파이프라인 작성자의 자기 보고다. LangChain의 52.8→66.5는 특정 모델·벤치마크에서 여러 하네스 변경을 함께 적용한 결과다. 두 수치 모두 이번 구현의 예상 개선률이 아니다. [1] [3]

07 / 실제 반영 위치

다음 개발에서 사용하는 경로

현재 관리 작업공간에 29개 파일을 반영했다. 기존 Git HEAD와 인덱스, 다른 세션의 작업공간은 변경하지 않았다. 새 기본 진입점은 아래 스크립트이며 운영자가 사용자를 대신해 호출한다.

기본 진입점

`/Users/woolee/benchmark/scripts/harness-run.sh`

별도 개발 worktree를 지정하고 원문과 계약을 보존한다. 실행기 설치 checkout 자체를 개발 workspace로 사용하지 않는다.

코드·문서	담당 기능
`scripts/harness_runtime/requirements.py`	계약 구조·원문 인용·준비 상태·검토 스키마
`scripts/harness_runtime/controller.py`	SQLite 상태, 단계 실행, 완료 판정, 변경·재개
`scripts/harness_runtime/process.py`	Codex 프로세스, 실행 로그, timeout·취소, 제한 명령
`scripts/harness_runtime/cli.py`	draft / init / run / status / change / feedback
`AGENTS.md` · 역할 TOML 7개	새 운영 방식과 단계 담당자 지침. 기존 절차의 충돌 정리
`docs/harness/external-control.md`	실행 예시·변경 기록·중단/재개·운영 한계

운영자의 작업 순서

draft에서 초안을 만들거나 직접 계약을 작성한다. 중요한 질문을 해소한 뒤 init으로 원문·계약을 저장하고 run으로 명세 검토부터 실행한다. status / events는 상태와 실제 이력을 보여 준다. 문제는 feedback, 승인된 후속 범위는 change에 기록한다.

QUICK/FULL은 새 독립 요청에서 한 번 선택하고 후속 작업은 재사용한다. 실행기의 개발·검토 기본 모델은 gpt-6.1-sol, 추론 medium이며 --model로 선택할 수 있다. 현재 운영 대화 모델은 고정하지 않는다. 옛 work-flow는 새 상태 변경을 차단하고 이력 조회 경로로 남겼다.

08 / 평가와 남은 한계

사용하면서 무엇을 확인하면 좋은가

이번 구현은 사용자의 지시에 따라 실제 개발을 진행하며 평가하는 방식을 따른다. 별도 테스트 과제를 통과해야 사용할 수 있는 구조로 다시 만들지 않았다. 평소 개발에서 아래 현상이 보이면 당시 요청과 작업 ID를 함께 남기면 수정 방향을 더 정확히 정할 수 있다.

사용 중 볼 지점	확인할 실제 경험
추상적 요청의 시작	바로 구현하는 대신 중요한 빈칸을 구체화하는가? 이미 정한 정책을 반복해서 묻지는 않는가?
요구 범위 보존	명시한 기능이 계약·구현까지 이어지는가? "간단히", "나중에"라는 판단으로 빠지지는 않는가?
완료의 근거	실제 관찰·실행 결과와 남은 조건을 보여 주는가? 실패를 완료로 바꾸지는 않는가?
개선의 실용성	문제를 설명했을 때 해당 지침·검증·구체화 단계가 바뀌는가? 작은 수정에 절차가 과도하지 않은가?

현재 확인한 것과 아직 확인하지 않은 것

확인한 것은 Python 7개 파일의 문법, 런타임 모듈 4개의 import, TOML 8개·JSON 3개의 파싱, 셸 문법과 CLI 도움말 연결, 적용 전후 파일 지문이다. 모델 호출·검증 명령·중단/재개·샌드박스 경계 프로브는 이번 작업에서 실행하지 않았다.

적용 범위: 외부 실행기를 거친 작업의 공식 완료 상태를 제어한다. native 대화의 모든 final 응답을 가로채는 전역 훅은 아니다. 운영 지침으로 새 진입점을 사용하도록 연결했지만, 다른 실행 경로까지 자동 감시하는 것은 아니다.

검증의 한계: 코드 지문은 의존성·외부 데이터까지 고정한 완전한 환경 재현물이 아니다. 읽기 전용 검증에 맞는 로컬 fixture와 산출물 경로 준비가 필요하다. 검토 모델도 누락을 놓칠 수 있고, 테스트가 부실하면 잘못된 구현이 통과할 수 있다.

권한의 한계: 사용자 메시지와 계약 지문을 연결한 receipt는 운영 기록이다. 같은 OS 계정의 비제한 프로그램까지 인증·차단하는 권한 서버는 아니다. 현재 설계와 실제 사용 결과를 함께 보고 필요한 부분부터 수정한다.

09 / 참고 자료

원문 링크와 읽을 위치

확인일: 2026-10-28. 번호와 제목을 누르면 원문이 열린다. 논문은 특정 실험 조건의 연구 결과, 공식 글은 설계 경험·사용법, 커뮤니티 글은 작성자의 경험담으로 구분해 읽는다.

[1] u/Prize-Individual4729 / 커뮤니티
I tried letting AI orchestrate AI. Here's why I switched to deterministic workflows.
2026년 4월 게시글 · 본문 및 개발 워크플로 댓글

[2] Anthropic · Prithvi Rajasekaran / 공식 기술 글
Harness design for long-running application development
2026-03-24 · Planner와 Evaluator 설명

[3] LangChain · Vivek Trivedy / 공식 기술 글
Improving Deep Agents with harness engineering
2026-02-17 · Build & Self-Verify, Trace Analyzer

[4] Adam Fourney 외 · Microsoft Research / 논문
Magentic-One: A Generalist Multi-Agent System for Solving Complex Tasks
arXiv:2411.04468v1 · 2024-11-07 · 4.1절, 5.3절

[5] Jiahang Lin 외 / 논문
Agentic Harness Engineering: Observability-Driven Automatic Evolution of Coding-Agent Harnesses
arXiv:2604.25850v4 · 3절, 4.4.2절, Limitations

[6] Anthropic / 공식 기술 글
Building effective agents
2024-12-19 · Workflows / Agents, Evaluator-optimizer

[7] Microsoft Learn / 공식 문서
Agentic application patterns
Deterministic workflows / Agent-directed workflows

[8] LangChain / 공식 문서
Thinking in LangGraph
노드·공유 상태·전환과 오류 처리 설계

[9] GitHub / 공식 오픈소스
Spec Kit
명세 중심 개발 및 구조화된 프로세스 · 관련 방식 대조 자료

구현 근거: 로컬 scripts/harness_runtime/ 및 docs/spec/agentic-workflow.md. 사용자 직접 적용 결정과 이전 방식 비교는 docs/decisions/2026-10-28-external-harness-control.md에 보존했다.